

RESEARCH

Open Access



A retrieval-augmented LLM framework for severity prediction of bug reports in cloud-based mobile applications

Asif Ali¹, Izhar Ahmed Khan², Yuanqing Xia^{3,1*}, Yuan Tian⁴, Saba Sajid⁵ and Mohammed Alsuhaibani⁶

Abstract

The effective maintenance of Mobile Applications (MAPPs) relies on the timely and accurate triage of bug reports, where severity labels determine the order of issue resolution. Manual severity assessment is often labor-intensive, error-prone, and inconsistent, highlighting the need for automated approaches that capture both semantic and contextual information. This study introduces RAG-GPT-SBR, a Retrieval-Augmented Generation framework that leverages a fine-tuned GPT-2 model for automated bug severity prediction in Cloud-based MAPPs. In addition to textual content, the framework incorporates non-textual metadata such as device type, operating system, crash signature, and bug frequency. Each new report is enriched with semantically related historical cases retrieved from a curated knowledge base using a hybrid BM25 and dense retrieval mechanism, forming structured input prompts for GPT-2. The fine-tuned model encodes this composite information and generates contextual embeddings, which are fed into a classification head to predict severity levels. Experiments conducted on a public Hugging Face bug report dataset demonstrate that RAG-GPT-SBR outperforms existing deep learning baselines, achieving gains of 7.98%, 8.20%, 7.90%, and 8.10% in accuracy, precision, recall, and F1-score, respectively. The results affirm that combining retrieval-augmented context with generative language models enhances reliability, scalability, and consistency in automated mobile application maintenance.

Keywords Retrieval-augmented generation, Cloud-based Mobile applications (CMAPPs), Deep Learning (DL), Classification, LLM

*Correspondence:

Yuanqing Xia
xia_yuanqing@bit.edu.cn

¹School of Automation, Beijing Institute of Technology, Beijing 100081, China

²College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China

³Zhongyuan University of Technology, Zhengzhou, Henan 450007, China

⁴School of Computer Engineering, Nanjing Institute of Technology, Nanjing, China

⁵School of Optics and Photonics, Beijing Institute of Technology, Beijing 100081, China

⁶Department of Computer Science, College of Computer, Qassim University, Buraydah, Saudi Arabia

Introduction

Cloud-based mobile applications (CMAPPs) play a pivotal role in the software development life cycle (SDLC), as they support reliable operation, enhance usability, and enable continuous improvement after deployment. A central component of this stage involves the timely identification and resolution of user-reported bugs, which is essential for maintaining application quality, user satisfaction, and long-term system stability. Fixing bugs not only ensures a timely and seamless user experience but also mitigates issues that could lead to operational disruptions or reputational damage [1]. To manage this process efficiently, developers rely on bug-tracking platforms

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

such as JIRA [2], which streamline the reporting, organization, and triaging of bugs throughout the software development life cycle [3].

The bug reports reported via these systems usually have an abundance of information on the defects that users experience, including some contextual factors that could have involved their occurrence. A typical bug report contains a number of important characteristics including the severity, summary, priority, the date of reporting, unique identifier as well as a textual description of the problem. Among these elements, severity is particularly critical because it guides the prioritization of bug resolution efforts. In Bugzilla [4], for example, severity levels range from trivial and minor to normal, major, critical, and blocker. However, accurately assigning severity levels is often challenging, as it requires domain knowledge and experience. Inexperienced users may misjudge the urgency of an issue, potentially delaying essential fixes and highlighting the importance of expert input in severity classification.

Numerous automated approaches have been developed to predict bug severity, with most relying on machine learning (ML) and deep learning (DL) techniques [5–8]. Although these methods have shown promising performance, they generally depend on textual features or shallow representations that capture limited contextual information. Recent studies further highlight the importance of sentiment cues in severity prediction [9, 10]. For example, bug reports that contain negative affective-related language, such as “pathetic interface”, “dark”, or “problematic,” are often linked to higher severity bug reports [11, 12]. Sentiment features combined with historical information on resolved bugs can significantly improve bug severity predictions for CMAPPs in terms of accuracy, context, and robustness.

To address these limitations, we propose RAG-GPT-SBR, a Retrieval-Augmented Generation–Enhanced GPT-2 framework for automated bug report severity prediction. In contrast to the traditional methods of encoder, RAG-GPT-SBR combines retrieval-augmented reasoning with non-textual contextual features to achieve a higher accuracy in making predictions in CMAPPs. The framework starts with the preprocessing of bug report, sentiment extraction, text normalization among other preparation processes. Each preprocessed report is then augmented with semantically related historical reports retrieved from a curated knowledge base using a hybrid approach that combines lexical matching (BM25) and dense semantic retrieval. The retrieved evidence, along with the original report text and non-textual metadata (e.g., device type, operating system version, and crash signature), is structured into input prompts for GPT-2. A fine-tuned GPT-2 model, equipped with a classification head over the pooled representations, is trained to

predict the severity level of each report. During inference, the model jointly uses retrieval-based evidence and contextual metadata to generate accurate severity predictions. Experimental results demonstrate that RAG-GPT-SBR consistently outperforms traditional ML and DL baselines, including BERT-based models [13]. To the best of our knowledge, this is the first framework to show that combining retrieval-augmented reasoning with a large language model (LLM) significantly improves automated bug severity classification in CMAPP maintenance.

The contributions of this study can be summarized as follows:

- We propose a novel RAG-Enhanced GPT-2 framework (RAG-GPT-SBR) for automated prediction of bug report severity in CMAPP maintenance. To the best of our knowledge, this is the first implementation that integrates RAG with a generative LLM for this task.
- The RAG-GPT-SBR model combines both textual content and non-textual contextual features (e.g., device type, OS, crash metadata), as well as semantically relevant past bug reports from a knowledge store, to achieve improved severity prediction performance.
- Empirical evaluation demonstrates that RAG-GPT-SBR achieves significant performance improvements over traditional baselines, with increases of 7.89% in accuracy, 8.20% in precision, 7.90% in recall, and 8.10% in F1-score. Overall, the proposed framework consistently outperforms conventional approaches in automated bug severity classification, establishing its effectiveness and reliability for maintenance tasks in Cloud-based Mobile Application Programs (CMAPPs).

The remainder of this article is organized as follows: related work on bug report severity prediction (including machine learning, deep learning, and retrieval-augmented approaches) is discussed in Section “[Literature review](#)”. A more detailed description of the proposed RAG-GPT-SBR framework is presented in Section “[Proposed approach](#)”. Section “[Evaluation](#)” includes the experimental setup, evaluation metrics, results, and discussions of potential threats to validity. Section “[Conclusions](#)” concludes the paper and offers directions for future research on automated CMAPPs bug triaging.

Literature review

Bug reporting in CMAPPs is typically classified according to severity level, which ranges from critical to trivial. Critical bugs are the most severe bugs and often lead to system crashes or data loss, while trivial bugs are typically an inconvenience or visual inconsistency from the user’s

perspective. The proper allocation of severity is essential in order to rank bugs and allocate resources properly in the software maintenance processes. Scholars have put forward a few automated techniques, relying on ML, and DL, which use textual content, metadata, and contextual data of the bug reports. Even though these approaches have shown good outcomes, they usually just use the information included in this bug report. This results in them frequently ignoring historical knowledge and non-textual contextual characteristics that would help substantially with the accuracy of classification, especially of higher level of severity or criticality topics.

ML-based severity prediction

Proper severity of bug report prediction is one of the most crucial issues in maintenance of software and mobile applications because it directly determines the distribution of resources and its priorities to be updated in time. The use of ML methods to fully automate this classification has become a research topic of large importance in recent years to remove the drawbacks of manual triaging, since it is both slow and inconsistent. The first automated method that was implemented for severity prediction of bug reports was developed by Menzies and Marcus [14] in the SEVERity Issue Assessment (SEVERIS) system. SEVERIS defined detailed severity levels similar to NASA's classification to support prioritization and used an ML classifier built on feature vectors formed from the top k most important words. Later, Haering, Lamkanfi, and colleagues [8, 15] extended this to open-source repositories, examining textual descriptions from major projects such as Eclipse, GNOME, and Mozilla. To make prediction easier, these studies often grouped severity labels into broader classes, usually severe and non-severe, to enable automated triage.

Later studies explored the use of traditional ML techniques for bug analysis. Alenezi and Banitaan [16] employed Naive Bayes (NB), Decision Trees (DT), and Random Forests (RF) to predict bug priority, evaluating two feature sets: one based on term-frequency-weighted words from bug descriptions and another combining operating system metadata with severity information. Their findings showed that the second feature set improved performance, with RF and DT outperforming NB, which itself achieved better results than earlier keyword-based approaches. Similarly, Gujral et al. [17] analyzed Eclipse bug reports and attained 72% precision and 69% accuracy by constructing custom dictionaries tailored for component-specific severity prediction. Zhang et al. [18] also conducted similar work analyzing over 27306 bug reports to classify them, achieving better results with multinomial NB (MNB), than other classification approaches, such as standard NB, k Nearest Neighbor (KNN), and SVM classifier approaches.

Tian et al. [19] assessed the K-Nearest Neighbors (KNN) algorithm on more than 65,000 Bugzilla reports with the objective of performing severity prediction. Pooja [20] applied Support Vector Machines (SVM) to rank crash bug reports in Firefox by implementing entropy-based measures. Kumari et al. [21] introduced new entropy-based metrics that capture the uncertainty inherent in bug report datasets and tested new metrics on PITS and Mozilla, showing that their new metrics outperformed prior models. Yang et al. [22] created an emotion-word dictionary to extract sentiment from bug reports and built a new emotionally-aware algorithm called EWD Multinomial Naïve Bayes (EWD MNB), which was found to outperform traditional MNB and related models [23–26].

DL-based severity prediction

Recent studies [27–30] have shown the strong impact of DL in fields such as speech recognition, sentiment analysis, and computer vision, [31–34]. However, its use for automated bug severity prediction is still limited. Even so, several researchers have made important progress in this area.

Yu et al. [35] developed a neural network-based framework for priority prediction of bug reports for a large international healthcare industry. The system was tested on five different products by using a three-fold cross-validation and showed clear gains in results such as precision, accuracy, and recall. Sharma and Abi Kanaan et al. [33, 36] designed a priority prediction model using several ML methods, including SVM, NB, KNN, and NN, to classify new bug reports. Their results showed that these traditional models could correctly predict the priority of less than 70% of bugs reports in the OpenOffice and Eclipse projects [37, 38].

Recent research has increasingly emphasized the role of AI and LLM in reshaping software engineering and CMAPPs education and practice, providing insights into both human-AI collaboration and real-world development challenges. Maryam et al. [39] examined the integration of Large Language Models (LLMs) in software engineering education, identifying 25 motivators and 30 demotivators influencing their adoption in Finnish higher education. The study highlighted the potential of LLMs to support personalized learning and automation, alongside challenges related to ethics, privacy, and academic integrity. Similarly, Khan et al. [40] analyzed 13,903 developer discussions from Stack Overflow to explore real-world software development practices. Their findings revealed 15 discussion themes and 49 challenges, emphasizing project management, domain-driven design, and software requirements as key focus areas for advancing software engineering research and practice.

Ali et al. [13] developed a DL-based approach for the prediction of bug severity using BERT. Their method incorporated natural language processing models for preprocessing and computed sentiment scores for bug reports, employing sentiment analysis tools such as SentiWordNet [41], SentiStrengthSE [42], SentiCR [43], Senti4SD [44], and EmoTxt [45]. Among these, Senti4SD was found to be the most effective for software bug reports. The incorporation of sentiment features within a BERT framework led to a significant enhancement of 40.43% in precision, which serves as the SoTA¹ baseline for automated severity prediction.

To summarize, previous studies examined various ML and DL-based approaches for severity prediction of bug reports. However, the majority of existing studies use only textual data from traditional software projects, not from CMAPPs. Additionally, studies have difficulty exploiting implicit feature representations or historical knowledge of bugs in the prediction process. The proposed RAG-GPT-SBR framework addresses these gaps by integrating RAG with GPT to enhance contextual understanding and reasoning. Unlike traditional ML models such as SVM, RF, or LR, which depend on handcrafted features, and earlier DL models like CNN or LSTM that capture limited context, RAG-GPT-SBR dynamically retrieves semantically relevant historical bug reports during inference to augment the input representation. This retrieval mechanism enhances contextual understanding, mitigates data sparsity, and improves generalization across diverse projects. Consequently, the proposed

framework establishes a more robust, knowledge-driven, and context-aware paradigm for severity prediction in CMAPPs.

Importance of proposed research

To further underscore the significance of automated bug severity prediction, a statistical evaluation of the literature in the R programming environment was executed since the bibliography information (*bibtex*) was extracted from 146 articles from WoS² that were identified from 2010 through 2024, which represents a high degree of relevance. The bibliographic corpus also included 105 sources (i.e. 75 journal articles, 71 conference proceedings, and one review paper), and a total of the 2670 references were included by 385 authors. A statistical analysis was conducted, which utilized this dataset to produce co-occurrence networks, thematic maps, and visualizations of thematic evolution illustrated in Fig. 1, respectively.

The findings indicate that the severity prediction of bug reports in software development has been the focus of academic interest for many years, pointing to the importance of user reviews and defect reporting in software maintenance. But, early research often targets traditional desktop or enterprise software, while CMAPPs were rarely included in the focus of these works. The advent of mobile platforms has made the maintenance of CMAPPs more widespread. Particularly, CMAPPs maintenance concerns some domains of software that differ from traditional forms, such as platform diversity, frequent updates, and user platform diversity. Thus, the

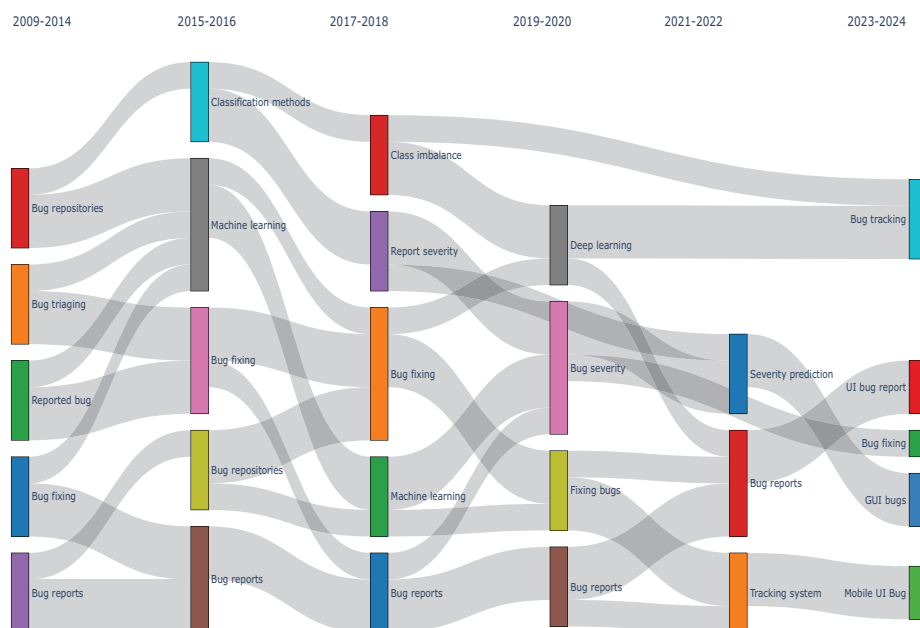


Fig. 1 Thematic evolution

¹ State of The Art.

² web of science.

accurate and automated severity prediction of CMAPPs defect reports is an important part of both maintenance and user satisfaction.

Proposed approach

Overview

The framework presents a Retrieval-Augmented Large Language Model (RAG-LLM) for autonomously determining the severity of bug reports. Although Deep Learning (DL) and transformer-based models (e.g., BERT) have achieved high performance in the area of text classification, the contextual reasoning capabilities of these models present limitations, especially in instances where the reasoning requires a perspective of historical knowledge and domain-specific context. Hence, by combining a RAG model with a fine-tuned GPT-2 to provide both the semantic understanding of the text and retrieval-augmented contextual reasoning to enhance the accuracy and robustness of severity classification tasks.

The process is divided into two main components. The first component is Knowledge Retrieval and Fusion, where bug reports are returned from a previously constructed knowledge base by performing a dense vector similarity search. The second component is LLM-Based Severity Prediction, where the retrieved context is concatenated with the cleaned and tokenized bug report, which is then passed to a fine-tuned GPT-2 model to classify the bug reports into one of the following severity levels: Critical, High, Medium, Low, and Trivial.

Mathematically, the process can be summarized as a function:

$$S = f_{\text{RAG-LLM}}(r_i, K) \quad (1)$$

where r_i refers to the i -th bug report, K refers

to the knowledge base, and $S \in \{1, 2, 3, 4, 5\}$ represents a severity label (e.g., Critical, High, Medium, Low, and Trivial).

The proposed framework's novelty is derived from its dynamic retrieval mechanism that situates each input query in relation to historical or semantic knowledge, thus improving the accuracy of the severity prediction as well as its explainability.

Figure 2 presents an overview of the proposed RAG-GPT-SBR framework for automated severity prediction of CMAPPs bug reports. The workflow of RAG-GPT-SBR consists of the following steps:

- First, a publicly available dataset of CMAPPs bug reports is collected from Hugging Face³, providing both textual descriptions and associated metadata.
- Next, the sentiment score of each bug report is computed using the SentiWordNet repository explained in section “Sentiment score calculation”, capturing the affective cues expressed by reporters.
- Relevant historical bug reports are then retrieved from a curated knowledge store using a hybrid retrieval strategy that combines lexical similarity (BM25) with dense semantic embeddings.
- The retrieved evidence, bug report text, and non-textual contextual features (e.g., device type, OS version, crash metadata) are assembled into structured input prompts suitable for GPT-2 processing.
- A fine-tuned GPT-2 model, enhanced with a classification head, is trained on these prompts to predict the severity of each bug report.
- Finally, the trained RAG-GPT-SBR model is evaluated on a held-out test set to assess its performance in comparison with conventional ML and DL baselines.

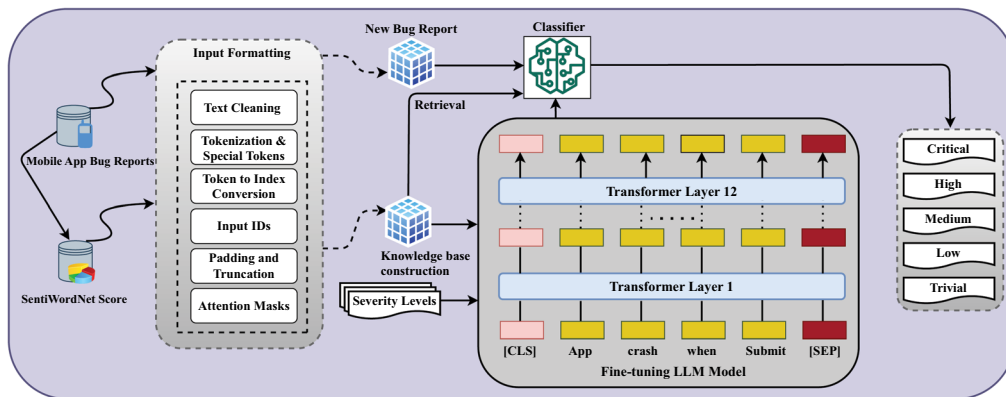


Fig. 2 RAG-GPT-SBR model

³https://huggingface.co/datasets/app_reviews/blob/main/app_reviews.py.

Illustrating example

The following example illustrates the workflow of the proposed RAG-GPT-SBR framework for predicting the severity of bug reports in CMAPPs. The example bug reports is collected from the Hugging Face dataset.

- Review = “The app crashes every time I try to upload a photo after the new update”
- Date = “September 15, 2023”
- Severity = “1”

In traditional classification models, bug reports are typically tokenized and fed directly into the model without additional contextual information. In contrast, the proposed RAG-LLM approach first retrieves semantically related bug reports, for example, “Crash on media upload due to memory leak” or “App instability post-update on Android 14” from a dedicated knowledge base. These retrieved documents are then concatenated with the original report to create an augmented input for GPT-2. This enriched context enables the model to make more informed inferences, such as identifying that the issue is likely to correspond to a critical severity level.

Let q denote the current bug reports and $D = \{d_1, d_2, \dots, d_n\}$ represent the knowledge base. The retrieval process aims to identify the subset of documents most semantically similar to the query. The degree of similarity between the query q and document d_j is defined by a semantic similarity function denoted as $\text{sim}(q, d_j)$ and it is computed typically in an embedding space derived by a pretrained encoder (e.g., FAISS-based dense vector embedding). The documents specific to a query are then chosen by observing the next criterion for displaying the top- k relevant documents d_k^* :

$$d_k^* = \arg \max_{d_j \in D} \text{sim}(E(q), E(d_j)), \quad (2)$$

where E denotes the embedding function that translates textual input into its relevant dense vector representation.

After the relevant documents are retrieved, the input sequence, I , is constructed by concatenating the bug reports with the retrieved documents such that

$$I = [q; d_1^*; d_2^*; \dots; d_k^*]. \quad (3)$$

The retrieval-augmented input is then passed to a fine-tuned GPT-2 model for severity classification. By conditioning the prediction on both the query and retrieved knowledge, the model captures the causal relationship between context (e.g., “crashes on upload”) and severity. The retrieval-augmented architecture adds to interpretability and substantially increases robustness

and generalization of severity prediction across many CMAPPs domains.

Problem definition

The collection of all bug reports are denoted by $R = \{r_1, r_2, \dots, r_N\}$. Each report r_i is assigned a severity label $S_i \in \{1, 2, \dots, C\}$, represents the total number of severity classes.

The severity assigning mapping function can be explained as:

$$f : R \times K \rightarrow S \quad (4)$$

such that:

$$\hat{S}_i = f(r_i, K) = \arg \max_S P(S | r_i, K) \quad (5)$$

In this case, the symbol $P(S | r_i, K)$ describes the posterior probability of the severity level given both the text input and the external retrieved knowledge.

This is framed as a multi-class classification task under retrieval-augmented generation, where contextually retrieving information modifies the representation space prior to classification.

The objective of the learning is to minimize the cross-entropy loss:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log \hat{y}_{i,c} \quad (6)$$

where $y_{i,c}$ is the true label and $\hat{y}_{i,c}$ is the predicted probability.

With this formulation, the adapted GPT-2 model learns from the syntactic structure of bug reports while also utilizing semantic information from retrieved documents to enhance classification robustness and interpretability.

Sentiment score calculation

Severity often reflects a strong intensity of sentiment with negative sentiment typically reflecting high severity levels. To operationalize this, we calculated a senti-

ment score, $S_i^{(sent)}$, for each bug report before feeding it to the model, which uses SentiWordNet. This feature served as an added signal to help improve accuracy in classification.

Considering a bug report text $r_i = [w_1, w_2, \dots, w_T]$, the sentiment score is computed as:

$$S_i^{(sent)} = \frac{1}{T} \sum_{t=1}^T \sigma(W_s \cdot h_t + b_s) \quad (7)$$

where h_t is the contextual embedding of token w_t produced by a sentiment encoder that has been pre-trained (e.g., GPT-2), W_s and b_s are trainable parameters, and σ is the sigmoid activation function.

The sentiment scores are normalized between 0 and 1, with values close to 1 being strongly negative sentiment. The sentiment information is concatenated with the token embeddings prior to passing into the LLM:

$$X_i = [E(r_i); S_i^{(sent)}] \quad (8)$$

This combined representation provides the model the ability to capture both semantic context and emotional intensity, both of which are important signals of severity in user-generated bug reports.

Input formatting

The efficacy of the RAG-LLM process is highly reliant on the input formatting and the retrieval augmentation process. This section describes the complete pipeline consisting of text cleaning, tokenization, adding special tokens, sequence processing, and incorporating knowledge into the pipeline.

Text cleaning

Unprocessed user bug reports are often characterized by the inclusion of emojis, URLs, hashtags, and non-standard text patterns. Each bug report r_i , therefore, is pre-processed as follows:

$$r_i' = \text{clean}(r_i) \quad (9)$$

In this cleaning function we performed removal of punctuation, emojis, and HTML tags, lowered cased all characters, lemmatized the words, and removed stopwords. Mathematically we can define text cleaning as:

$$r_i' = \{w_t \in r_i \mid w_t \notin \text{StopWords}\} \quad (10)$$

Word tokenization

After cleaning, each bug report is tokenized using GPT-2 byte-pair encoding (BPE):

$$T_i = \text{Tokenizer}(r_i') \quad (11)$$

This step converts words to a sequence of token IDs while maintaining subword-level granularity, enabling the model to handle rare and previously unseen terms that are common in user-generated content.

Addition of special tokens

The GPT-2 tokenizer employs special tokens which are inserted into the token sequence to aid the model

in processing the sequence at both training and test time. A beginning-of-sequence token, denoted [CLS], indicates the start of the input text, while the end-of-sequence token, denoted [SEP], indicates the end of the input text. The purpose of these tokens is to enable the model to contextualize the input, and therefore act as a tool for generating successful predictions, such as in the task of severity classification, which is discussed below. The [SEP] token also means “separator” and is used to separate any two sentences or sequences that are together placed in the input token sequence, marking the boundary between two sentences or sequences for the model during a task, such as when doing sentence pair classification.

The sequence of tokens with special tokens can be denoted as:

$$I_i = [\text{CLS}] + t_1 + t_2 + t_3 \dots + t_n + n' + [\text{SEP}] \quad (12)$$

Token to index conversion

Upon including special tokens, each of the input sequence's tokens will be transformed into a numerical token identifier, usable by the model. Token-to-index conversion turns the textual tokens into their respective vocabulary indices according to the vocabulary set in the GPT-2 tokenizer. The enhanced tokenized input sequence after special token addition be represented as $I_i = [t_1, t_2, \dots, t_n]$, where each t_j is one of the tokens taken from the model's vocabulary V .

The function id maps each token t_j to its unique integer index $v_j \in \mathbb{N}$, which is defined as:

$$v_j = \text{id}(t_j), \quad \forall t_j \in I_i. \quad (13)$$

Therefore, the entire input sequence is transformed into an indexed format:

$$I_i^{(id)} = [v_1, v_2, \dots, v_n], \quad (14)$$

where $v_j \in [0, |V| - 1]$, and $|V|$ is the vocabulary size of the GPT-2 model.

This transformation is essential because neural networks process only numerical tensors, not raw text. At each time step, the integer index of each token v_j is used as a lookup key into the model's embedding matrix, where $W_E \in \mathbb{R}^{|V| \times d_{emb}}$, which return the corresponding dense embedding vector e_j for that token. In formal notation:

$$e_j = W_E[v_j], \quad (15)$$

where $e_j \in \mathbb{R}^{d_{emb}}$ and d_{emb} is the embedding dimension. Then, the entire sequence of embedding vectors

$E_i = [e_1, e_2, \dots, e_n]$ is forwarded into the model's subsequent layers in order to encode the contextual information through the transformer layers.

This mapping of token to index provides a consistency in number representation between the raw text inputs and the internal parameters of the model. In addition, the mapping maintains the interpretability of each token identity from the vocabulary of the GPT-2, allowing fine-tuning and retrieving representation ability to be replicated across different iterations and machines.

Input IDs

The order of token index is then transformed into the corresponding sequence of input IDs, which form the ultimate numerical representation that is provided to the GPT-2 model. The input IDs are taken from the fixed vocabulary and can be represented as:

$$I = [101, id_1, id_2, \dots, id_n, 102] \quad (16)$$

In this context, I represents the entire series of input IDs for the bug report r , id_i indicates the i -th input ID in the series of input IDs for the bug report r , and n denotes the count of input IDs for the report. Special tokens [CLS] and [SEP] signify the beginning and end of the sequence, respectively, which is considered structural information necessary for the GPT-2 model when predicting severity.

Padding and truncation

To accommodate for efficient batch processing, we pad or truncate the length of the input sequences to a fixed maximum length. In this implementation the maximum sequence length is 402 tokens (the maximum length of the longest bug report), if it has fewer than 402 tokens it is padded with a token id (ID=0), if it is longer than 402 tokens, it would be truncated. This provides for a consistent input size for all the bug reports, which allows the RAG-GPT-SBR model to be trained and tested in a stable manner.

$$I'_{1:402} = \begin{cases} I_{1:j} [PAD]^{402-j} & \text{if } j < 402 \\ I_{1:402} & \text{if } j > 402 \end{cases} \quad (17)$$

In this context, I' represents the final sequence of input IDs for bug report r after padding and truncation have been applied to guarantee that all sequences are of the same predefined maximum length for the model input.

Attention Masks (AM)

The Attention Mask (AM) delineates the genuine tokens from the padding within an input. The differentiation is significant because the attention mechanism in the transformer uses the mask to identify which content to place attention on, and padding would be ignored. The AM is represented by a binary sequence, with 1s indicating the real tokens and 0s indicating padding. Given an input that has a length of n , but is padded to a length of m ; the Attention Mask for an input would be represented by a binary vector which includes a value of 1 for the first n positions from I (the original tokens), and a value of 0 for the remaining $m - n$ positions (the padding). The AM M_I' for the input sequence I is represented mathematically as:

$$M_{I'} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \dots & 1 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}$$

In the above matrix representation, rows of 1s represent the locations of real tokens in the input sequence, while rows of 0s represent padding tokens. This enables the transformer model to allocate computation only to the meaningful tokens and adequately ignore the padded ones.

The pre-trained GPT-2 model creates contextualized embedding vectors for each bug report that also maximize the information content and preserve semantic and syntactic structures. In subsequent RAG and downstream severity prediction classifier stages, the embeddings are used with the integrations of non-text and sentiment scores. Table 1 shows the example bug report in Section “[Illustrating example](#)”, with embeddings, to demonstrate how the text and auxiliary information are used to provide a meaningful severity classification.

Knowledge base construction

The knowledge base (K) is the contextual store that allows the LLM to augment its reasoning beyond its pre-trained parameters. The knowledge base consists of a heterogeneous collection of textual artifacts that includes historical bugs from bug reports and developer release bugs involving mobile applications. Each bug report $r_j \in K$ is embedded into a high-dimensional dense vector representation by a sentence-level embedding model (e.g., Sentence-BERT), which translates the semantics of each textual artifact into an embedding $v_j = E(r_j)$ of the text. The embeddings served to organize documents that were semantically close in the embedding space into proximate spaces, facilitating retrieval based on similarity measures.

Table 1 Text embedding example

Details	Embeddings
Example	[0.0027, -0.0031, 0.0019, -0.0065, 0.0035, ...]

In order to enhance the efficiency of retrieval, all embeddings are placed in a vector index structure (e.g., FAISS), which allows for approximate nearest neighbor searches in sub-linear time. The knowledge base is semantic and computation-efficient for real-time retrieval during prediction. The knowledge base is both complete and diverse, providing the model access to relevant prior examples of similar bugs, patterns of errors, and solutions that effectively ground the model's predictions in history and domain knowledge.

In formal terms, the knowledge is represented by the set $K = \{(r_j, v_j)\}_{j=1}^n$, which satisfies r_j referring to the bug reports and $v_j = E(r_j)$ its corresponding vector embedding. This known pair structure allows for an easy similarity calculation in the retrieval procedure, i.e., each query will be supplemented with the associated most relevant contextual information.

Retrieval-augmented generation

Once the knowledge base has been constructed, the retrieval-augmented generation process begins by incorporating external contextual information into the severity prediction pipeline. For a new bug report r_i , the first step is for the retrieval component to encode it with the same embedding function as was used for the knowledge base producing a query vector $q_i = E(r_i)$. The system then performs a semantic similarity search between q_i and all the embeddings of the documents $v_j \in K$. The similarity score is computed as follows:

$$\text{sim}(q_i, v_j) = \frac{q_i \cdot v_j}{\|q_i\| \|v_j\|}, \quad (18)$$

which captures the angular closeness between two vectors in the embedding space. The retrieval component then identifies the top- k similar bug reports according to the maximization of this similarity measure, giving rise to the retrieved bug report set $R_i = \{r_1^*, r_2^*, \dots, r_k^*\}$.

The retrieved text is then concatenated with the original cleaned and tokenized bug report to give rise to the augmented input sequence $I_i = [r_i; R_i]$. This augmentation improves the contextual input with domain-specific references to allow the model to reason beyond the local text of the review itself. Giving the GPT-2 model relevant

previous examples and descriptions of similar bugs using the RAG process successfully transforms the prediction task into a context-aware inference problem.

After passing through augmentation, the input sequence is encoded into token embeddings and then passed through the GPT-2 architecture that outputs hidden contextual representations at multiple layers of transformers. The output hidden representation denoted as h_{I_i} , is then passed through a classification head with a softmax output layer that gives the probability distribution for the possible severity classes:

$$P(S | I_i) = \text{Softmax}(W_h \cdot h_{I_i} + b_h), \quad (19)$$

where learnable parameters are W_h and b_h . This probability distribution is the model's confidence on the severity classes. Since both retrieval and generation are used in the framework, this takes advantage of explicit knowledge retrieval and implicit reasoning. This makes predictions of severity more accurate and explainable.

Fine-tuning of GPT-2

The GPT-2 model is adapted for the severity prediction task through fine-tuning on a labeled dataset of bug reports. The hyperparameters setting of the proposed approach explained in Table 2. In the fine-tuning process the model adjusts its generative representations in order to align with the supervised classification goal. Each input sequence I_i (the inputs being the bug reports, retrieved knowledge and sentiment score) is inputted into GPT-2 and outputs final contextual embedding h_{I_i} , which is mapped to the label space via a linear projection layer followed by softmax activation. This in turn provides the conditional probability distribution given by $P(S | I_i)$.

The fine-tuning is therefore performed in order to minimize a joint loss function which balances the accuracy of the classification with the alignment of the retrieval. The total loss is defined as:

$$L_{\text{total}} = L_{\text{CE}} + \lambda L_{\text{retrieval}}, \quad (20)$$

where L_{CE} denotes the cross-entropy loss in relation to the predicted and actual severity labels, and $L_{\text{retrieval}}$ is

Table 2 Hyperparameters setting for the proposed approach

Samples	Maximum Length	BS	LR	Epsilon	AH	HS	Input Shape
40000	402	16	2×10^{-05}	1.00×10^{-08}	12	768	[40000,402]
	256	16	2×10^{-05}	1.00×10^{-08}	12	768	[40000,256]
	128	8	2×10^{-05}	1.00×10^{-08}	12	768	[40000,128]
4000	402	16	2×10^{-05}	1.00×10^{-08}	12	768	[4000,402]
	256	16	2×10^{-05}	1.00×10^{-08}	12	768	[4000,256]
	128	8	2×10^{-05}	1.00×10^{-08}	12	768	[4000,128]

a contrastive loss that encourages semantic alignment of query embeddings and the embeddings of the documents being retrieved. The hyperparameter λ controls the trade-off for these two objectives.

During training, the parameters of the GPT-2 model and the retrieval encoder are updated together using backpropagation, allowing the model to learn to classify severity levels accurately, but also to improve evidence retrieval by returning documents that are semantically relevant, and better contextualize the output. Fine-tuning continues until convergence, typically measured in validation loss and F1-score.

The RAG-based GPT-2 model that has been fine-tuned demonstrates the ability to produce severity predictions that are grounded in context, leveraging previously existing work to extend large pre-trained transformer models to a level of domain adaptation. The proposed model combines the three stages of retrieval, generation, and optimization through supervision into a single architecture, as part of the process of improving the generalization performance and interpretability of conventional transformer based classifiers.

Evaluation

In this section, we first develop research questions to evaluate the efficacy and reliability of the RAG-LLM severity prediction framework. Next, we detail the process for collecting CMAPPs bug reports and describe the steps for data preprocessing, embedding generation, and retrieval-augmented prediction. We then establish the evaluation metrics, and describe the evaluation process to assess model performance. Finally, we summarize the main experimental results, discuss threats to validity, and explain how we addressed the threats to ensure the results were reliable and insights proven robust.

Research questions (RQ)

The assessment of the RAG-GPT-SBR severity prediction framework is based upon these research questions:

- **RQ1:** To what extent RAG-GPT-SBR model accurately predict the severity of CMAPPs bug reports?
- **RQ2:** How does RAG-GPT-SBR compare with traditional ML, DL, and baseline approaches in predicting bug severity?
- **RQ3:** In what ways has RAG improved GPT-2-based feature extraction over traditional embedding methods like Word2Vec?
- **RQ4:** What did sentiment analysis add to the overall performance of the RAG-GPT-SBR severity prediction framework?

Research Question 1 assesses the degree to which the RAG-GPT-SBR framework can predict the severity of CMAPPs bug reports. For the benchmarking of the severity prediction model, we compare RAG-GPT-SBR performance with baseline approaches like random prediction and zero-rule algorithms. Random prediction also creates a prediction of outcomes by randomly selecting any value from the training data, and it is often used to set a baseline when there are no prior baseline, especially when severity classes are rare or underrepresented. The zero-rule algorithm is used to classify into the most common severity class in the data set, and is the simplest form of heuristic baseline.

RQ2 compares RAG-GPT-SBR against traditional ML and DL approaches, used to assess how effectively it can predict the bug report severity, compared to methods that do not augment GPT-2 with RAG. The third research question describes the benefits of GPT-2-based feature extraction over traditional embeddings like Word2Vec. Word embeddings modeled using GPT-2 enhanced by RAG retrieval have better contextual understanding and produce better severity predictions, when compared to Word2Vec embeddings that were provided as input to a recurrent neural network (RNN) model. Finally, RQ4 interrogates the role of sentiment information in severity prediction.

Dataset

The historical dataset of app reviews is taken from Hugging Face⁴, compiled by Giovanni Grano (University of Zurich). It provides monolingual English reviews and also includes features such as package name, review text, date, and star rating. A summary of the dataset can be found in Table 3.

This dataset provides a comprehensive overview of Android apps across 23 different categories, and contains information on user reviews and their associated app metrics over time. The dataset includes nearly 370 CMAPPs downloaded from the F-Droid repository,

Table 3 Dataset detail

CMAPPs Reviews	
Total No. of CMAPPs Reviews	40000
CMAPPs Review Types	5
Rating	Count# (%)
1 Star	7500 (18.75%)
2 Star	2950 (7.38%)
3 Star	4650 (11.62%)
4 Star	7550 (18.87%)
5 Star	17350 (43.37%)
Words in CMAPPs Review	
Minimum Word count	1
Maximum Word count	422

⁴<https://huggingface.co/datasets>.

comprised of 550 apps and app versions. Of the nearly 265,000 user reviews, 40,000 were extracted using specialized text-mining techniques for the RAG-GPT-SBR experiments.

Table 3 shows that the selected dataset comprises 40000 reviews labeled according to severity: “1 star”, “2 star”, “3 star”, “4 star”, and “5 star”. Specifically, the distribution includes 7500 reviews (18.75%) with “1 star”, 2950 (7.38%) with “2 star”, 4650 (11.62%) with “3 star”, 7550 (18.87%) with “4 star”, and 17350 (43.37%) with “5 star”, as illustrated in Fig. 3. The length of reviews ranges from 1 word to 422 words.

Figure 4 reveals a word cloud representation of the frequently occurring terms, in the CMAPPs reviews. The more frequently terms occur, the larger and bolder they are presented, whereas the least frequently occurring terms are presented smaller and less boldly. This corpus represents a rich dataset for training and evaluating the RAG-GPT-SBR model to learn semantic relationships and retrieval of bug report histories for improved predictions of severity.

Evaluation metrics

To evaluate the performance of the RAG-GPT-SBR severity prediction model, we employ standard metrics for multi-class classification, namely, A_{cc}^5 , P_r^6 , R_c^7 ,

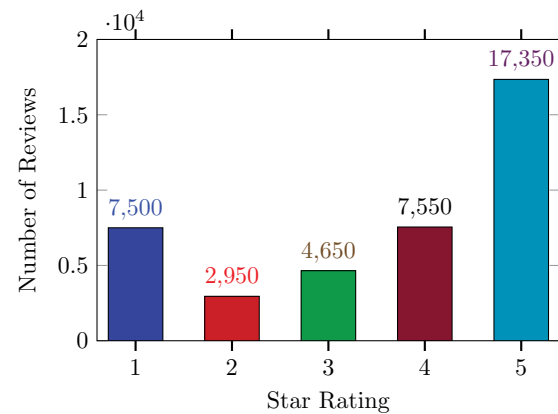


Fig. 3 Severity breakdown by class

and F1_s⁸. The employed metrics quantify the extent to which the model correctly predicts the severity level of app reviews, whilst considering the potential class imbalances and the distribution of the levels of severity

$$A_{-cc} = \frac{TP + TN}{TP + TN + FP + FN} \quad (21)$$

$$P_{-r} = \frac{TP}{TP + FP} \quad (22)$$

$$R_c = \frac{TP}{TP + FN} \quad (23)$$

⁵ Accuracy.

⁶Precision.

⁷Recall.

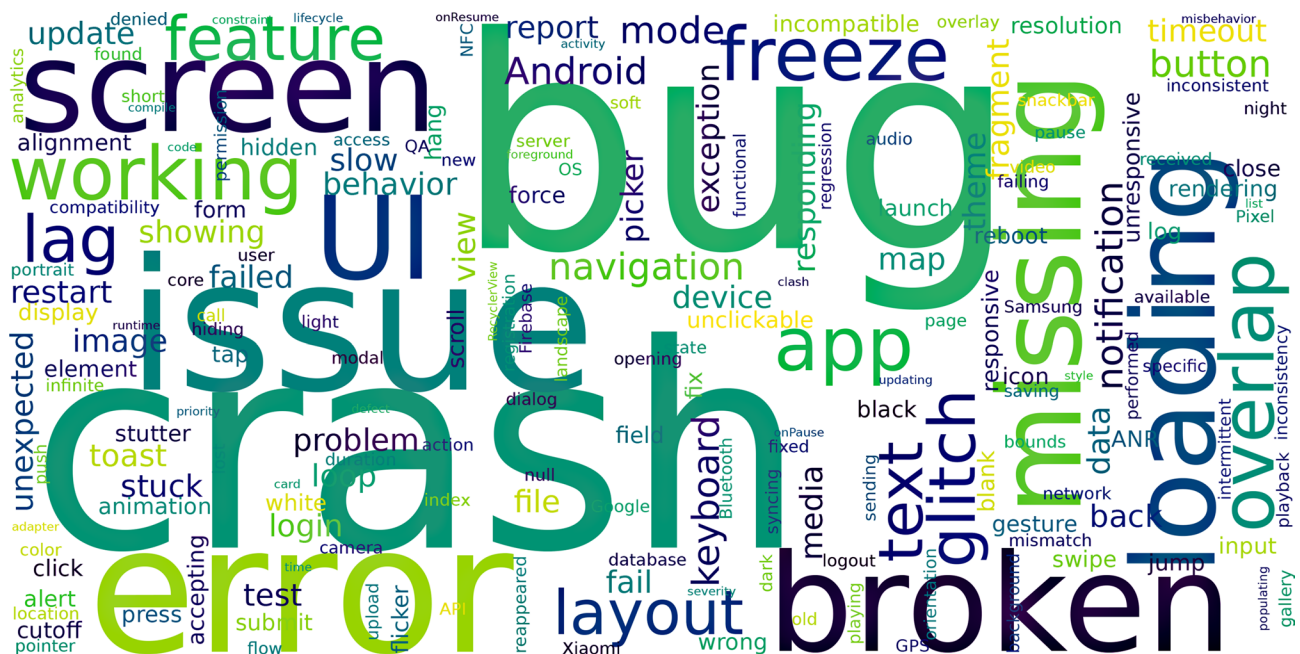
⁸F1-score.

Fig. 4 Common terms in app reviews

$$F1_s = \frac{2 \times P_r \times R_c}{P_r + R_c} \quad (24)$$

Within these metrics, True Positive (TP) refers to the number of app reviews accurately assigned to their actual star rating, and True Negative (TN) refers to the number of reviews accurately classified into not belonging to a star class. False Positive (FP) counts reviews incorrectly assigned into a star rating class, while False Negative (FN) counts reviews that were incorrectly not classified as belonging to star rating class.

Process

The evaluation of the RAG-GPT-SBR severity prediction models follows a set of steps. The first step is collecting app reviews, denoted as R in this study, from the Hugging Face repository. The textual data is then preprocessed using tokenization and semantic embedding methods suitable for retrieval-augmented generation. Preprocessing includes sentence tokenization, punctuation removal, lowercasing, stop word removal, and adding special tokens for GPT-2 input formatting. Each review is converted into token IDs and then adjusted to a consistent length through padding and truncation.

In the next step, we apply a hold-out validation strategy, dividing the dataset into 70% for training and 15% for validation, and 15% for testing. The training set contains historical reviews, while the testing set includes more recent reviews to reflect real-world deployment scenarios. The Hold-out validation process is chosen over k -fold cross-validation due to the use of past data to predict severity in new incoming app reviews. To address class imbalance, the Synthetic Minority Over-sampling Technique (SMOTE) was used on the training set. This approach generates synthetic samples for the minority classes, preventing the model from being biased toward the majority class and avoiding data leakage from the validation and test sets.

For model training, the RAG framework combines GPT-2 as the generator with a retrieval component that fetches semantically similar historical bug reports from a knowledge base. The retrieved reports are merged with the input review and fed into GPT-2 for contextual severity prediction. Fine-tuning adjusts the model parameters using the training set to minimize prediction error. The model's performance is then evaluated on the testing set using standard metrics, including A_cc , P_r , R_c , and

$F1_s$, to provide a reliable assessment of severity prediction for CMAPP reviews.

To validate the RAG-GPT-SBR severity prediction model, we follow these steps:

- First, the training review set is selected from the dataset.
- These training reviews are used for fine-tuning the RAG-GPT-SBR model, which combines GPT-2 as the generator with a retrieval component for contextual information.
- At the same time, these training review set are also used for training baseline approaches, including LSTM, Adaptive Boosting (AdaBoost), Neural Network (NN), CNN, SVM, LR, RF, RNN, and DT.
- Next, the trained RAG-GPT-SBR model and the baseline models are used to predict the severity of each review in the testing set.
- Finally, the performance of each model is evaluated using standard metrics, including A_cc , P_r , R_c , and $F1_s$, to measure their effectiveness in predicting severity.

Results evaluation

A_cc of RAG-GPT-SBR

To answer RQ1, we have compared the proposed RAG-GPT-SBR model with two baseline models: zero-rule and random prediction. These baselines provide reference points to evaluate the performance of our approach. To the best of our knowledge, RAG-GPT-SBR is the first model specifically designed for mobile app review severity prediction. Consequently, no direct baselines exist for comparison in this domain. Therefore, these baseline algorithms and BERT-based severity prediction of bug reports [13] are used to offer a meaningful assessment of the RAG-GPT-SBR model's accuracy and effectiveness.

Table 4 shows the evaluation results of the proposed RAG-GPT-SBR model alongside the random prediction and zero-rule baseline methods. Columns 2–5 report A_cc , P_r , R_c , and $F1_s$ for each method, with row representing a specific classifier and corresponding result. The results of hold-out validation in Table 4 present a clear comparison of the proposed model performance with baseline approaches.

Results show that the RAG-GPT-SBR model achieves A_cc of 98.40%, P_r of 98.48%, R_c of 98.32%, and $F1_s$ of 98.40%. In comparison, the zero-rule baseline achieves an A_cc of 56.85%, P_r of 31.66%, R_c of 39.39%, and an $F1_s$ of 39.39%, while the second baseline, random prediction, achieves an A_cc of 33.69%, P_r of 32.62%, R_c of 29.53%, and an $F1_s$ of 31.00%. These findings highlight the superior performance of the RAG-GPT-SBR model in predicting the severity of app reviews.

Table 4 Comparison of RAG-GPT-SBR against baseline approaches

Model	A_cc (%)	P_r (%)	R_c (%)	$F1_s$ (%)
RAG-GPT-SBR	98.40	98.48	98.32	98.40
Zero Rule	56.85	31.66	52.45	39.39
Random Prediction	33.69	32.62	29.53	31.00

Observations

The following observations can be made from Table 4:

- The RAG-GPT-SBR significantly outperforms all baseline models.
- In terms of A_{cc}, P_r, R_c, and F1_s, the RAG-GPT-SBR improvement over zero-rule and random prediction is $((98.40\% - 56.85\%) / 98.40\% \times 100\% = 42.22\%$ and $((98.40\% - 33.69\%) / 98.40\% \times 100\% = 65.76\%)$, $((98.48\% - 31.66\%) / 98.48\% \times 100\% = 67.85\%$ and $((98.48\% - 32.62\%) / 98.48\% \times 100\% = 66.83\%)$, $((98.32\% - 52.45\%) / 98.32\% \times 100\% = 46.65\%$ and $((98.32\% - 29.53\%) / 98.32\% \times 100\% = 69.95\%)$, and $((98.40\% - 39.39\%) / 98.40\% \times 100\% = 59.96\%$ and $((98.40\% - 31.00\%) / 98.40\% \times 100\% = 68.40\%)$, respectively. RAG-GPT-SBRs predictive accuracy comes from the ability to draw contextual embeddings from the text and retrieval-augmented knowledge contained in the app reviews to augment the background knowledge task.
- RAG-GPT-SBR outperformed the zero-rule and the random prediction classifier to collect information on users' experience based on generative language modeling, retrieval of augmented contextual reasoning to the background knowledge task, using the GPT-2 embeddings and external knowledge part to collect contextual and layered meanings, and semantics and institutional knowledge involved in the app review. The architectural design of RAG-GPT-SBR facilitates a more effective understanding of complex relationships in the information contained in the app review texts, compared to both simplistic baseline and non-contextual approaches.

Table 5 Comparison with different ML, DL and baseline models based on GPT-2 embeddings

Model Name	A _{cc} (%)	P _r (%)	R _c (%)	F1 _s (%)
RAG-GPT-SBR	98.40	98.48	98.32	98.40
BERT [13]	91.13	91.02	91.13	91.03
SevPredict [46]	76.20	78.60	78.60	78.60
LR	57.12	53.36	55.76	54.53
SVM	54.98	42.64	54.71	47.66
RF	55.72	46.44	56.38	50.92
AdaBoost	54.12	32.74	28.58	30.52
DT	42.12	26.72	24.68	25.66
MNB	66.44	78.56	72.22	75.26
NB	66.92	88.66	72.38	79.70
BNB	68.28	89.28	69.48	78.14
NN	58.27	41.98	29.18	34.43
CNN	58.18	38.74	36.56	37.62
RNN	59.26	34.84	32.26	33.50
LSTM	56.68	34.28	32.64	33.44

Performance comparison with ML, DL, and baseline models

To address RQ2, we evaluate a range of widely used classification algorithms, including SVM, DT, RF, RNN, LSTM, NN, CNN, NB, MNB, Bernoulli NB, LR, AdaBoost, and DT, using embeddings generated by RAG-GPT-SBR. The results of these algorithms are then compared with the fine-tuned RAG-GPT-SBR. Parameter tuning is performed for each algorithm to identify the optimal settings. Each model is trained and evaluated across different parameter configurations to determine the most effective setup. Finally, the performance of all algorithms is compared using their best-performing configurations.

The results of the RAG-GPT-SBR, SVM, RF, AdaBoost, DT, RNN, LSTM, NN, CNN, and LR are presented in Table 5. Each row of Table 5 has a specific classifier, while columns 2–5 presents A_{cc}, P_r, R_c, and F1_s, respectively. The fine-tuned RAG-GPT-SBR model demonstrates superior performance with an A_{cc} of 98.40%, P_r of 98.48%, R_c of 98.32%, and F1_s of 98.40%. In contrast, the RNN achieves an A_{cc} of 59.26%, P_r of 34.84%, R_c of 32.26%, and F1_s of 33.50%. The MNB, NB, and BNB classifiers report A_{cc} of 66.44%, 66.92%, and 68.28%, P_r of 78.56%, 88.66%, and 89.28%, R_c of 72.22%, 72.38%, and 69.48%, and F1_s of 75.26%, 79.70%, and 78.14%, respectively. LSTM achieves an A_{cc} of 56.68%, P_r of 34.28%, R_c of 32.64%, and F1_s of 33.44%. The NN classifier records an A_{cc} of 58.27%, P_r of 41.98%, R_c of 29.18%, and F1_s of 34.43%, while CNN obtains an A_{cc} of 58.18%, P_r of 38.74%, R_c of 32.56%, and F1_s of 35.38%. The LR shows an A_{cc} of 57.12%, P_r of 53.36%, R_c of 55.76%, and F1_s of 54.53%. SVM achieves 54.98% A_{cc}, 42.64% P_r, 54.71% R_c, and 47.66% F1_s. RF exhibits 55.72% A_{cc}, 46.44% P_r, 56.38% R_c, and 50.92% F1_s. AdaBoost achieves 54.12% A_{cc}, 32.74% P_r, 28.58% R_c, and 30.52% F1_s. Lastly, DT records 42.12% A_{cc}, 26.72% P_r, 24.68% R_c, and 25.66% F1_s.

Observations

Based on the results described in Table 5 and Fig. 5, the following observations can be made about the performance of RAG-GPT-SBR:

- RAG-GPT-SBR outperforms all other baseline approaches, such as RF, AdaBoost, DT, RNN, LSTM, NN, CNN, LR, and SVM. This advantage comes from its use of retrieval-augmented generation, which combines pre-trained GPT-2 language modeling with relevant external knowledge. The model generates highly contextual and semantically rich representations of app reviews, and its attention mechanisms improve understanding of long-range dependencies and complex textual relationships.

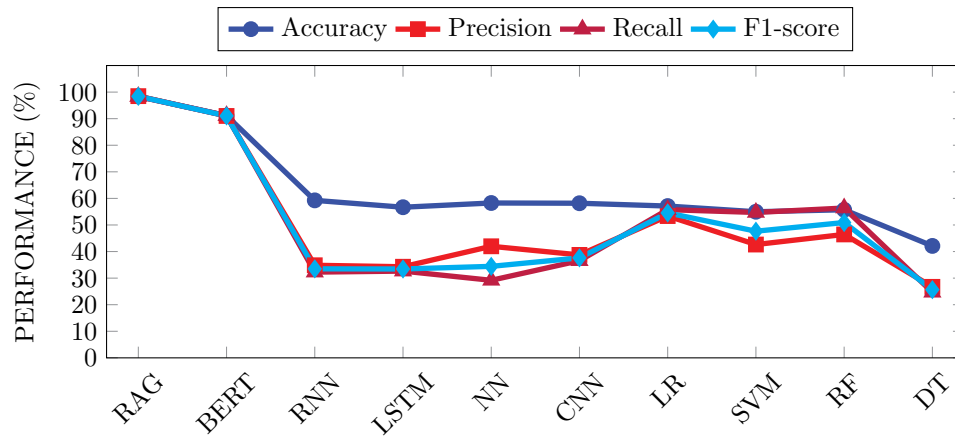


Fig. 5 Comparison of different ML and DL models (in %) based on GPT-2 embeddings

Table 6 Embeddings influence on results

Feature Modeling	A_cc (%)	P_r (%)	R_c (%)	F1_s (%)
With GPT-2 (CNN)	58.18	38.74	36.56	37.62
With Word2Vec (CNN)	52.16	33.98	31.28	32.57

- DL models (RAG-GPT-SBR, BERT, NN, CNN, RNN, and LSTM) consistently outperform ML approaches (AdaBoost, RF DT, LR, SVM). Their ability to learn hierarchical and complex representations from textual data reduces the need for manual feature engineering and enables effective handling of large-scale datasets.
- Among traditional classifiers, LR demonstrates relatively better performance in P_r, R_c, and F1_s. This is likely because LR efficiently captures linear relationships and remains robust against imbalanced data. However, its effectiveness still depends on the dataset and task context.
- Although NB, MNB, and BNB achieve slightly higher precision (1.86%, 1.78%, and 2.03%), RAG-GPT-SBR surpasses them in A_cc (37.97%, 34.49%, and 36.61%), R_c (33.13%, 32.48%, and 35.39%), and F1_s (15.59%, 15.30%, and 16.63%). The high precision but low recall of these Naive Bayes variants shows that they struggle with the imbalanced dataset, producing many false positives and negatives, while RAG-GPT-SBR provides balanced and robust classification.

Influence of embeddings

To thoroughly investigate RQ3, we conducted a comparison of RAG-GPT-SBR embeddings against Word2Vec embeddings with the CNN model, as summarized in Table 6. The first column indicates the embedding type, while columns 2–5 report A_cc, P_r, R_c, and F1_s for each approach. When using RAG-GPT-SBR embeddings, CNN achieves an A_cc of 58.18%, P_r of 38.74%, R_c of 32.56%, and F1_s of 35.38%. In comparison, with Word2Vec embeddings, CNN reaches an A_cc of 52.16%, P_r

Table 7 The influence of SentiWordNet on results

Sentiment Score Computation	A_cc (%)	P_r (%)	R_c (%)	F1_s (%)
With SentiWordNet	98.40	98.48	98.32	98.40
With Senti4SD	98.18	98.22	98.16	98.18
Without SentiWordNet	98.12	98.06	98.08	98.07

of 33.98%, R_c of 31.28%, and F1_s of 32.57%. The findings indicate that the contextual and retrieval-augmented semantic information contained in the RAG-GPT-SBR embeddings better informs the model, resulting in improved severity prediction performance.

Observations

From Table 6, the following points can be observed:

- Using embeddings generated by the pre-trained RAG-GPT-SBR model, the CNN classifier shows improvements of 6.02% in A_cc, 4.76% in P_r, 5.28% in R_c, and 5.05% in F1_s compared to Word2Vec embeddings. These results demonstrate the advantage of retrieval-augmented embeddings in enhancing the CNN model's performance. RAG-GPT-SBR outperforms Word2Vec by combining a transformer-based generative model with retrieval-augmented context. While Word2Vec relies only on local word co-occurrences, GPT2 incorporates external knowledge through retrieval, capturing richer semantic relationships and contextual information. As a result, CNN with GPT2 embeddings achieves higher accuracy and more reliable severity predictions for app reviews.

Influence of sentiwordnet

To answer RQ4, we compare the performance of RAG-GPT-SBR with and without the integration of SentiWordNet. Table 7 presents the evaluation results, where the first column specifies the input configuration, and

columns 2–5 report A_{cc}, P_r, R_c, and F1_s. On average, RAG-GPT-SBR with SentiWordNet and Senti4SD achieves A_{cc}, P_r, R_c, and F1_s of 98.40%, 98.48%, 91.32%, and 98.40%, and 98.02%, 97.76%, 97.24%, and 97.50%, respectively. In comparison, RAG-GPT-SBR without SentiWordNet achieves slightly lower values of 98.12%, 98.06%, 98.08%, and 98.07%. These results show that incorporating sentiment-aware features from SentiWordNet provides a consistent, though modest, improvement in severity prediction when combined with the retrieval-augmented RAG-GPT-SBR embeddings.

Observations

From Table 7, the following points can be noted:

- The results show that by adding SentiWordNet to the RAG-GPT-SBR model produces small but consistent improvements in performance compared to the version without SentiWordNet. Specifically, it increases average A_{cc}, P_r, R_c, and F1_s by 0.28%, 0.42%, 0.24%, and 0.33%, respectively. This integration assists the model in more accurately capturing sentiment and contextual information in bug reports, with a slight boost in sentiment-aware severity prediction. Even though these are incremental improvements, they illustrate the benefit of using additional lexical resources for a deeper understanding of natural language. The use of Senti4SD as a substitute sentiment resource also increases all metrics, A_{cc}, P_r, R_c, and F1_s by 0.06%, 0.16%, 0.08%, and 0.11%, respectively.

In summary, RAG-GPT-SBR has been shown to perform well to predict CMAPP bug report severity, and the inclusion of sentiment-aware features provides small but consistent gains.

Research implications and threats to validity

Research implications

This study contributes to the growing body of software engineering research by introducing the RAG-GPT-SBR model, which leverages RAG for intelligent software bug report analysis and prioritization. The framework advances the understanding of how large language models, when combined with structured retrieval mechanisms, can enhance contextual reasoning and reduce misclassification in software maintenance workflows. From a research standpoint, the proposed approach provides a foundation for exploring hybrid architectures that integrate generative and discriminative capabilities for software analytics. It also opens new avenues for investigating multimodal and cross-domain bug triaging, where textual and code-level information can be fused for deeper semantic interpretation. Moreover, this study

extends the methodological discourse by validating RAG-based models as a viable alternative to traditional deep learning techniques for context-aware software quality assurance tasks.

Practical implications

From a practical perspective, the RAG-GPT-SBR model offers a robust and scalable solution for improving the efficiency and accuracy of real-world software maintenance processes. By integrating retrieval-based contextual knowledge with generative reasoning, the model supports developers in identifying, classifying, and prioritizing bug reports with higher precision, thereby reducing resolution time and improving software reliability. Software organizations can adopt this approach to automate initial triaging stages, optimize resource allocation, and enhance communication between developers and testers. Additionally, the framework can be integrated into existing issue-tracking and project management tools, providing decision support for software maintenance teams in large-scale and distributed development environments. The deployment of RAG-GPT-SBR thus has the potential to significantly improve productivity and consistency in software engineering practice.

Threats to validity

The selection of evaluation metrics is a potential source of threat to construct validity. Although A_{cc}, P_r, R_c, and F1_s are routinely utilized within the field, and we also utilize these metrics to facilitate comparative purposes with previous studies, these metrics may inadvertently affect the way results are interpreted when using RAG-GPT-SBR.

The use of *SentiWordNet* to compute sentiment in bug reports is another potential source of threat to construct validity. Although SentiWordNet fits our dataset, consideration should be given to the use of different sentiment resources or lexical repositories that might affect performance results for RAG-GPT-SBR.

A threat to internal validity is partly due to the implementation of our baseline and comparative approaches. After establishing and meticulously validating these approaches, we nonetheless acknowledge the possibility of undetected implementation errors or bias.

Two primary threats to external validity include generalizability of our findings as we focused on CMAPPs bug reports for evaluation purposes. While we applied RAG-GPT-SBR on bug reports in this context, applying it to bug reports in other domains or projects would almost certainly yield different results. Additionally, consideration should be given to the models performance being affected by hyperparameter configurations such as training dataset size and tuning of other learning parameters, which may impact RAG-GPT-SBRs overall performance.

Conclusions

The automation of bug report classification and severity assessment is vital to the efficient maintenance of Cloud-based Mobile Applications (CMAPPs) through the prioritization of critical bug reports. The manual severity classification of bug reports is generally slow, inconsistent, and prone to variability and bias, particularly with large-scale CMAPPs and significant amounts of data. To address this we proposed RAG-GPT-SBR, an automated bug severity prediction framework that employs a RAG component and a fine-tuned GPT-2 model. The framework begins with the use of sentiment analysis to capture the emotional tone of the bug reporters to aid in severity estimation. Each report is then tokenized using GPT-2 preprocessing strategies such as padding, truncation, and additional special tokens. Each report is encoded to arrive at a uniform sequence. To add more context, relevant historical bug reports are retrieved and augmented as part of the existing context, creating embeddings that preserve both semantic information and emotional tone. The embeddings then pass through a fine-tuned GPT-2 classifier, helping it to learn how to associate each of the textual, emotional, and contextual features to specific predicted severity levels. Through experimental evaluation RAG-GPT-SBR improved accuracy, precision, recall, and F-measure by 7.98%, 8.20%, 7.90%, and 8.10%, respectively, compared to the baseline methods. Future work will extend this framework to support multi-lingual bug reports and incorporate multimedia elements like screenshots and logs, enabling richer, cross-modal understanding of bug severity in mobile application maintenance.

Acknowledgements

The researchers would like to thank the Deanship of Graduate Studies and Scientific Research at Qassim University for financial support (QU-APC-2025). This work was supported by the Beijing Natural Science Foundation Haidian Original Innovation Joint Fund Project under Grant L252035 and the National Natural Science Foundation of China under Grant number W2533166. Also, by the Jiangsu Province Engineering Research Center of IntelliSense Technology and System.

Author contributions

Asif and Izhar wrote the main manuscript text, Xia and Tian worked as supervisor of the project, Saba and Asif develop the experimental process, and Mohammed provide the funding for the research work.

Funding

The researchers would like to thank the Deanship of Graduate Studies and Scientific Research at Qassim University for financial support (QU-APC-2025). This work was supported by the Beijing Natural Science Foundation Haidian Original Innovation Joint Fund Project under Grant number L252035 and the National Natural Science Foundation of China under Grant number W2533166. Also, by the Jiangsu Province Engineering Research Center of IntelliSense Technology and System.

Data availability

No datasets were generated or analysed during the current study.

Declarations

Ethical approval

Not applicable.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare no competing interests.

Received: 24 October 2025 / Accepted: 4 December 2025

Published online: 29 December 2025

References

1. Moran K (2015) Enhancing android application bug reporting. In Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. ESEC/FSE 2015, ACM, New York, NY, USA, 1045–1047. <https://doi.org/10.1145/2786805.2807557>
2. Atlassian: JIRA Issue Tracker
3. Xia X, Lo D, Wang X, Zhou B (2013) Accurate developer recommendation for bug resolution. In 2013 20th Working Conference on Reverse Engineering (WCRE), pp 72–81. <https://doi.org/10.1109/WCRE.2013.6671282>
4. Mozilla: Bugzilla Issue Tracker
5. Ramay WY, Umer Q, Yin XC, Zhu C, Illahi I (2019) Deep neural network-based severity prediction of bug reports. IEEE Access 7:46846–46857. <https://doi.org/10.1109/ACCESS.2019.2909746>
6. Sharma M, Kumari M, Singh RK, Singh VB (2014) Multiattribute based machine learning models for severity prediction in cross project context. In: Computational science and its applications – ICCSA 2014. Springer, Cham, pp 227–241
7. Chaturvedi K, Singh V (2012) Determining bug severity using machine learning techniques. In Software Engineering (CONSEG), 2012 CSI Sixth International Conference On, IEEE, pp. 1–6
8. Lamkanfi A, Demeyer S, Soetens QD, Verdonck T (2011) Comparing mining algorithms for predicting the severity of a reported bug. In 2011 15th European Conference on Software Maintenance and Reengineering, pp 249–258. <https://doi.org/10.1109/CSMR.2011.31>
9. Umer Q, Liu H, Sultan Y (2018) Emotion based automated priority prediction for bug reports. IEEE Access 6:35743–35752. <https://doi.org/10.1109/ACCESS.2018.2850910>
10. Chen G, Peng S, Zeng R, Hu Z, Cao L, Zhou Y, Ouyang Z, Nie X (2022) P-norm broad learning for negative emotion classification in social networks. Big Data Min Analytics 5(3):245–256. <https://doi.org/10.26599/BDMA.2022.9020008>
11. Rahman MS, Reza H (2022) A systematic review towards big data analytics in social media. Big Data Min Analytics 5(3):228–244. <https://doi.org/10.26599/BDMA.2022.9020009>
12. Nti IK, Quarcoo JA, Aning J, Fosu (2022) G.K.: a mini-review of machine learning in big data analytics: applications, challenges, and prospects. Big Data Min Analytics 5(2):81–97. <https://doi.org/10.26599/BDMA.2021.9020028>
13. Ali A, Xia Y, Umer Q, Osman M (2024) Bert based severity prediction of bug reports for the maintenance of mobile applications. J Syst Softw 208:111898
14. Menzies T, Marcus A (2008) Automated severity assessment of software defect reports. In 2008 IEEE International Conference on Software Maintenance, pp 346–355. <https://doi.org/10.1109/ICSM.2008.4658083>
15. Haering M, Stanik C, Maalej W (2021) Automatically matching bug reports with related app reviews. In 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), pp 970–981. IEEE
16. Alenezi M, Banitaan S (2013) Bug reports prioritization: which features and classifier to use? In 2013 12th International Conference on Machine Learning and Applications, vol 2. pp 112–116. <https://doi.org/10.1109/ICMLA.2013.114>
17. Gujral S, Sharma G, Sharma S (2015) Classifying bug severity using dictionary based approach. <https://doi.org/10.1109/ABLAZE.2015.7154933>
18. Zhang T, Yang G, Lee B, Chan ATS (2015) Predicting severity of bug report by mining bug repository with concept profile. In Proceedings of the 30th

- Annual ACM Symposium on Applied Computing. SAC'15, ACM, New York, NY, USA, 1553–1558. <https://doi.org/10.1145/2695664.2695872>
19. Tian Y, Lo D, Xia X, Sun C (2015) Automated prediction of bug report priority using multi-factor analysis. *Empirical Softw Eng* 20(5):1354–1383. <https://doi.org/10.1007/s10664-014-9331-y>
 20. Choudhary P (2017) Neural network based bug priority prediction model using text classification techniques. *Int J Adv Res Comput Sci* 8(5):1315–1319. <https://doi.org/10.26483/jarcs.v8i5.3559>
 21. Kumari M, Sharma M, Singh VB (2018) Severity assessment of a reported bug by considering its uncertainty and irregular state. *Int J Open Source Softw Processes* 9:20–46. <https://doi.org/10.4018/IJOSSP.2018100102>
 22. Yang G, Baek S, Lee J-W, Lee B (2017) Analyzing emotion words to predict severity of software bugs: a case study of open source projects. In *Proceedings of the Symposium on Applied Computing. SAC'17*, ACM, New York, NY, USA, 1280–1287
 23. Khalajzadeh H, Shahin M, Obie HO, Agrawal P, Grundy J (2022) Supporting developers in addressing human-centric issues in mobile apps. *IEEE Trans Softw Eng* 49(4):2149–2168
 24. Yue C, Huang R, Towey D, Zhou L (2024) A median-based fuzzy approach to software quality evaluation. *Tsinghua Science and Technology*
 25. Wang H, Li Y, Yang J, Hu D, Liao Z (2024) Camdroid: context-aware model-based automated gui testing for android apps. *Tsinghua Sci Technol* 30(1):55–67
 26. Izadi M, Akbari K, Heydarnoori A (2022) Predicting the objective and priority of issue reports in software repositories. *Empirical Softw Eng* 27(2):50
 27. Oliaee AH, Das S, Liu J, Rahman MA (2023) Using bidirectional encoder representations from transformers (bert) to classify traffic crash severity types. *Nat Language Process J* 3:100007
 28. Xu S, Zhang C, Hong D (2022) Bert-based nlp techniques for classification and severity modeling in basic warranty data study. *Insur: Math Econ* 107:57–67
 29. Glani Y, Ping L, Shah SA, Ke L (2025) Ccdive: A deep dive into code clone detection using local sequence alignment. *Tsinghua Sci Technol* 30(4):1435–1456
 30. Saga T, Tanaka H, Iwasaka H, Nakamura S (2022) Multimodal prediction of social responsiveness score with bert-based text features. *IEICE Trans Inf Syst* 105(3):578–586
 31. Peng S, Zeng R, Liu H, Cao L, Wang G, Xie J (2023) Deep broad learning for emotion classification in textual conversations. *Tsinghua Sci Technol* 29(2):481–491
 32. Alyahyan S, Alatawi MN, Alnfai MM, Alotaibi SD, Alshammari A, Alzaid Z, Alwageed (2024) H.S.: software reliability assessment: an architectural and component impact analysis. *Tsinghua Sci Technol* 30(2):908–925
 33. Sharma M, Bedi P, Chaturvedi KK, Singh VB (2012) Predicting the priority of a reported bug using machine learning techniques and cross project validation. In *2012 12th International Conference on Intelligent Systems Design and Applications (ISDA)*, pp 539–545. <https://doi.org/10.1109/ISDA.2012.6416595>
 34. Ali A, Xia Y, Navid Q, Khan ZA, Khan JA, Aldakheel EA, Khafaga D (2024) Mobile-ui-repair: a deep learning based ui smell detection technique for mobile user interface. *PeerJ Comput Sci* 10:2028
 35. Yu L, Tsai W-T, Zhao W, Wu F (2010) Predicting defect priority based on neural networks. In: Cao L, Zhong J, Feng Y (eds) *Advanced data mining and applications*. Springer, Berlin, Heidelberg, pp 356–367
 36. Abi Kanaan M, Couchot J-F, Guyeux C, Laiymani D, Atechian T, Darazi R (2023) A methodology for emergency calls severity prediction: from pre-processing to bert-based classifiers. In *IFIP International Conference on Artificial Intelligence Applications and Innovations*, Springer, pp. 329–342
 37. Gomes L, Silva Torres R, Côrtes ML (2023) Bert-and tf-idf-based feature extraction for long-lived bug prediction in floss: a comparative study. *Inf Softw Technol* 160:107217
 38. Yu S, Fang C, Zhang Q, Cao Z, Yun Y, Cao Z, Mei K, Chen Z (2023) Mobile app crowdsourced test report consistency detection via deep image-and-text fusion understanding. In *IEEE Transactions on Software Engineering*
 39. Khan M, Akbar MA, Kasurinen J (2025). Integrating llms in software engineering education: motivators, demotivators, and a roadmap towards a framework for Finnish higher education institutes. *arXiv preprint arXiv:2503.22238*
 40. Khan AA, Khan JA, Akbar MA, Zhou P, Fahmideh M (2024) Insights into software development approaches: mining q & a repositories. *Empirical Softw Eng* 29(1):8
 41. Uddin J, Ghazali R, Mat Deris M, Naseem R, Shah H (2016) A survey on bug prioritization 47
 42. Islam MR, Zibran MF (2018) Sentistrength-se: exploiting domain specificity for improved sentiment analysis in software engineering text. *J Syst Softw* 145:125–146. <https://doi.org/10.1016/j.jss.2018.08.030>
 43. Ahmed T, Bosu A, Iqbal A, Rahimi S (2017) Senticr: a customized sentiment analysis tool for code review interactions. In *Proceedings of the 32Nd IEEE/ACM International Conference on Automated Software Engineering. ASE 2017*, IEEE Press, Piscataway, NJ, USA, 106–111
 44. Calefato F, Lanubile F, Maiorano F, Novielli N (2018) Sentiment polarity detection for software development. *Empirical Softw Engg* 23(3):1352–1382. <https://doi.org/10.1007/s10664-017-9546-9>
 45. Calefato F, Lanubile F, Novielli N (2017) Emotxt: a toolkit for emotion recognition from text. In *2017 Seventh International Conference on Affective Computing and Intelligent Interaction Workshops and Demos (ACIIW)*, pp 79–80
 46. Arshad MA, Riaz A, Fatima R, Yasin A (2024) Sevpredict: exploring the potential of large language models in software maintenance. *AI* 5(4):2739–2760. <https://doi.org/10.3390/ai5040132>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.